

RAPID: Robot Agentic Programming from Demonstrations for Nonprehensile Manipulation

Anonymous Authors

Abstract—This is the abstract.

I. INTRODUCTION

Recent advances in coding agents have introduced a promising paradigm in which executable and editable programs can be iteratively refined through interaction with the environment. This paradigm has recently begun to gain traction in robotic manipulation [1]–[4]. We argue that the central potential of coding agents for robotics lies not in one-shot code generation, but in the iterative process of generating, executing, verifying, and revising a program, which we refer to as the *agentic loop*. However, for this loop to operate, three ingredients must be available: (1) a *specification* that defines what the generated behavior should accomplish; (2) *primitives* that connect the generated program to low-level actions; and (3) an *interactive environment* in which the actions can be executed and its consequences observed, enabling the agent to verify whether the specification is satisfied. Existing robotic coding-agent systems instantiate this paradigm largely in settings where these ingredients are provided a priori, through human-engineered primitives, explicit task objectives, and resettable environments with reward or success feedback. This raises a fundamental question: **how can a coding agent close the agentic loop when these ingredients are not provided a priori?**

We explore *demonstrations* as a programming interface for addressing this challenge. Traditionally, programming a new manipulation behavior requires human engineers to specify both *how* the robot should interact with the environment and *what* outcome it should achieve, for example by implementing action primitives, task plans, reward functions, and success criteria. A demonstration provides a more direct way of communicating both: it shows a feasible way to accomplish the task while also conveying the intended result. Importantly, rather than merely reproducing the demonstrated trajectory, we aim to use this information to recover an executable and reusable manipulation program. However, demonstrations alone do not provide the interactive environment required to repeatedly test and refine the generated program, leaving the environment for verification as the remaining challenge in closing the agentic loop.

Motivated by this perspective, we introduce **Robot Agentic Programming from Demonstrations (RAPID)**, a framework for constructing and iteratively refining robot programs from demonstrations. Given a single demonstration and a language instruction, RAPID infers the task specification, constructs the required manipulation primitives, and composes them into a manipulation strategy. It further reconstructs the demonstrated

scene in simulation, providing an interactive environment for program verification. The coding agent can then execute the generated program, inspect failures, revise its solution, and repeat this process until the specification is satisfied. In this way, demonstrations provide the information required to *construct* the robot program, while simulation provides the interaction required to *verify and improve* it, allowing RAPID to automatically establish the three ingredients required to close the robotic agentic loop.

In this work, we apply this framework to *nonprehensile manipulation*, a domain in which the need for such an approach is particularly pronounced. Many prehensile manipulation tasks can be expressed using a small, well-established set of primitives: *pick*, *move in free space*, and *place*. Nonprehensile manipulation, in contrast, involves diverse modes of physical interaction, such as pushing, pivoting, toppling, flipping, and pressing, whose realization depends strongly on object geometry and contact conditions. Consequently, providing a coding agent with a sufficiently expressive set of primitives can itself require substantial engineering effort, as a human designer must identify and implement the interaction primitives needed for each task. Nonprehensile manipulation therefore provides a natural setting for studying whether a coding agent can construct its own manipulation interface from demonstrations, rather than relying on a library of manually engineered primitives.

A central challenge is ensuring that a program inferred from a demonstration can *generalize* beyond the demonstrated scene. Existing code-as-policy approaches can generate manipulation behaviors using concrete numerical values, such as moving an end-effector by a fixed distance or toward a fixed pose. Such programs are inherently tied to the demonstrated configuration and can therefore generalize poorly. RAPID instead represents each manipulation primitive as a *parameterized trajectory-optimization program*, whose parameterization and objectives are expressed using *object-centric and relational quantities*. Rather than encoding a particular motion numerically, the program specifies the geometric and contact relations that characterize the manipulation, while a trajectory optimizer compiles out the corresponding motion for the current scene. RAPID similarly composes primitives into a strategy through constraints over primitive parameters and object state variables, which are also expressed relationally. As a result, the generated program captures the invariant structure of the demonstrated strategy rather than scene-specific numerical values, allowing the same program to adapt to variations in object pose, size, and geometry.

The remaining challenge is *verification*: the agent needs an

environment in which it can execute and evaluate the programs inferred from demonstrations. RAPID addresses this challenge with a real-to-sim pipeline that integrates existing vision-language models and visual foundation models to reconstruct the demonstrated scene in simulation. Beyond reproducing the observed scene, RAPID also proposes plausible scene variants and evaluates the generated program across them, allowing the agent to explore how the strategy behaves under changes in the environment. The coding agent can then test candidate programs against the inferred task specification, diagnose failures, and iteratively revise the primitives and strategy. This simulation-based verification closes the agentic loop while encouraging the generated program to generalize beyond the demonstrated instance.

We evaluate RAPID on contact-rich nonprehensile manipulation tasks, where the system is given a single demonstration and must construct a reusable manipulation strategy. Our primary evaluation criterion is *generalization*: we test whether a program constructed from one demonstration can adapt to substantial variations in the environment, such as changes in object pose, geometry, physical parameters, and scene configuration, rather than merely reproducing the demonstrated behavior. We further evaluate the generated programs on physical robots and conduct ablation studies to assess the contribution of each component of RAPID.

Our contributions are threefold:

- We identify three key ingredients for closing the *agentic loop* in robotic manipulation: a task specification, action primitives, and an interactive environment. We introduce RAPID, which automatically establishes these ingredients from demonstrations.
- We introduce a program representation for reusable nonprehensile manipulation strategies, in which manipulation primitives are expressed as *parameterized trajectory-optimization programs* and composed through *object-centric and relational constraints*. This representation allows programs inferred from a single demonstration to adapt to variations in the environment rather than relying on scene-specific numerical values.
- We demonstrate that RAPID can construct and iteratively refine contact-rich manipulation programs from a single demonstration, achieving generalization across diverse variations in object geometry, pose, physical properties, and scene configuration, with successful transfer to physical robots.

II. RELATED WORK

**Agentic Programming for Robotics.
Learning from Few Demonstrations.
Nonprehensile Manipulation.**

III. PROBLEM STATEMENT

We consider programming a reusable *nonprehensile manipulation* strategy from a single demonstration D and a language instruction I . The demonstration is collected in one environment instance E_0 , while the resulting program should solve a family of task variants $\mathcal{E}$ that share the same task

objective but may differ in object geometry, pose, physical properties, and scene configuration.

We assume that no task-specific manipulation primitives, explicit success criterion, or interactive simulation environment are provided a priori. Given only (D, I) , the agent must first construct a set of executable manipulation primitives

$$\mathcal{P} = P_1, \dots, P_N,$$

and then compose these primitives into a strategy Π . The resulting program should capture the manipulation structure underlying the demonstration and generalize beyond the demonstrated instance:

$$(D, I) \longrightarrow (\mathcal{P}, \Pi), \quad \Pi(E; \mathcal{P}) \text{ succeeds for } E \in \mathcal{E}.$$

The goal is therefore not to reproduce the demonstrated motion, but to recover reusable primitives and a strategy that can adapt to unseen task variants.

IV. METHOD

Overview here.

A. Program Representation

1) Primitives as Parameterized Trajectory-Optimization Programs

2) Strategy as Relational Constraints over Primitives

B. Programming from Demonstration

1) Inferring the Task Specification

2) Constructing Primitives and Strategy

C. Real-to-Sim Reconstruction for Verification

1) Scene Reconstruction

2) Scene Variant Generation

D. Robot Agentic Programming Loop

V. EXPERIMENTS

VI. CONCLUSION

APPENDIX

REFERENCES

- [1] K. Chen, S. Xie, L. Fu, J. Yu, W. Pacini, S. Bajamahal, H. Kim, J. Drake, D. Kim, H. Xue, *et al.*, “GaP: A Graph-as-Policy Multi-Agent Self-Learning Harness For Variational Automation Tasks,” *arXiv:2607.05369*, 2026. **1**
- [2] W. Xiao, J. Xie, T. Zhang, H. Lin, L. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, *et al.*, “ENPIRE: Agentic Robot Policy Self-Improvement in the Real World,” *arXiv:2606.19980*, 2026. **1**
- [3] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, *et al.*, “ASPIRE: Agentic Skills Discovery for Robotics,” *arXiv:2607.00272*, 2026. **1**
- [4] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, D. Niu, F.-F. Li, *et al.*, “CaP-X: A framework for benchmarking and improving coding agents for robot manipulation,” *arXiv:2603.22435*, 2026. **1**

TABLE I: Experiment Results

Task	Skeleton	$\pi_{0.5}$	RAPID w/o Prim.	RAPID w/o Optim.	RAPID w/o Goal	RAPID w/o Rel.	RAPID
Store Box	push→push	–	–	–	–	–	–
	press→flip→push	–	–	–	–	–	–
	rotate→push→push	–	–	–	–	–	–
Grasp Box	push→flip→grasp	–	–	–	–	–	–
	push→flip→press→grasp	–	–	–	–	–	–
	press→push→flip→grasp	–	–	–	–	–	–
Pick up Book	tilt→grasp	–	–	–	–	–	–
	push→grasp	–	–	–	–	–	–

TABLE II: Experiment Results in the Real World

Task	Skeleton	RAPID
Store Box	push→push	–
	press→flip→push	–
	rotate→push→push	–
Grasp Box	push→flip→grasp	–
	push→flip→press→grasp	–
	press→push→flip→grasp	–
Pick up Book	tilt→grasp	–
	push→grasp	–